LESEFASSUNG · ALLE THEMEN

Git-Wissen kompakt

83 Aussagen mit Einordnung, 25 Antipatterns mit Handlungsempfehlungen und 24 Begriffe. Ohne Spiel und ohne Freischaltung vollständig nutzbar.

Mythos oder Fakt? Die Einordnung.

M01 · IRRTUM

GitHub ist öffentlich, GitLab ist privat.

Sowohl GitHub als auch GitLab unterstützen je nach Angebot private und öffentliche Repositorien (also verwaltete Projektverzeichnisse samt Versionsgeschichte). Entscheidend sind Sichtbarkeitseinstellungen, Benutzerrechte, Serverbetrieb, externe Kopien und der tatsächliche Umgang mit Daten. „Privat“ bezeichnet eine Zugriffseinstellung, keine umfassende datenschutzrechtliche Bewertung.

M02 · IRRTUM

Git ist GitHub.

Git ist die Versionsverwaltungssoftware; GitHub ist ein Dienst, der Git-Repositorien verwaltet und zusätzliche Funktionen anbietet. Gleiches gilt für GitLab und andere Anbieter. Git funktioniert auch ganz ohne Hosting-Plattform.

M03 · IRRTUM

Git ist zu komplex für die editorische Alltagsarbeit.

Grafische Bedienoberflächen sowie Integrationen in Arbeitsprogramme – etwa die Git-Unterstützung im Oxygen XML Editor – vereinfachen häufige Aufgaben wie Änderungen speichern, vergleichen und mit dem gemeinsamen Projektstand abgleichen. Grundbegriffe und ein sorgfältiger Umgang mit Konflikten bleiben wichtig; niemand muss dafür sämtliche Befehle kennen.

M04 · IRRTUM

Git ist ausschließlich ein Teamwerkzeug.

Auch allein lassen sich frühere Dateistände wiederherstellen, Experimente durchführen und der eigene Bearbeitungsverlauf nachvollziehen. Dies kann etwa bei Transkription und TEI-Auszeichnung hilfreich sein.

M05 · IRRTUM

Für Git braucht man einen Server und Internetzugang.

Git kann vollständig lokal auf einem einzelnen Rechner arbeiten. Erst für die Synchronisierung mit anderen Rechnern oder Menschen wird üblicherweise eine zusätzliche Kopie an anderer Stelle eingerichtet. Ein Commit ist dabei ein bewusst gespeicherter Versionsstand; dafür ist keine Netzwerkverbindung erforderlich.

M06 · IRRTUM

Git ist eine neutrale technische Lösung ohne Folgen für die Arbeitskultur.

Die Wahl von Abläufen, Zugriffsmöglichkeiten und Auswertungen beeinflusst, welche Tätigkeiten dokumentiert werden, wer Änderungen freigeben darf und wie Beiträge wahrgenommen werden. Aus einem Werkzeug können implizite Erwartungen an Arbeitsrhythmus und Rechenschaft entstehen.

M07 · IRRTUM

Eine gelöschte Datei ist aus Git verschwunden.

Frühere gespeicherte Versionsstände können die Datei weiterhin enthalten. Ein tatsächliches Entfernen aus der Versionsgeschichte verlangt besondere Maßnahmen und berücksichtigt noch nicht automatisch Kopien bei anderen Beteiligten.

M08 · IRRTUM

Wenn nur die Endfassung veröffentlicht wird, bleiben interne Kommentare verborgen.

Wer das Repositorium mitsamt Versionsgeschichte zugänglich macht, kann dadurch ältere Texte, interne Anmerkungen oder zeitweilig enthaltene Dateien offenlegen. Entscheidend ist der Unterschied zwischen der Publikation eines Dateiexports und der Publikation der Repository-Geschichte.

M09 · IRRTUM

Pseudonyme machen Git-Aktivitätsdaten anonym.

Personen können unter Umständen durch Kombination mit anderen Informationen wiedererkannt werden. Pseudonymisierung und Anonymisierung sind nicht gleichbedeutend; ob Identifizierbarkeit besteht, hängt vom konkreten Kontext ab.

M10 · IRRTUM

Öffentlich einsehbare Git-Daten dürfen beliebig personenbezogen ausgewertet werden.

Commit-Metadaten können personenbezogen sein. Eine Analyse muss gegebenenfalls unter anderem hinsichtlich Rechtsgrundlage, Zweck, Erforderlichkeit und Schutz der Betroffenen bewertet werden. Hinzu kommt: Solche Daten sind methodisch oft ungeeignet, Arbeitszeit oder Leistung direkt zu messen.

M11 · IRRTUM

Wer öffentlich an einem Repositorium mitarbeitet, stimmt jeder Auswertung zu.

Sichtbarkeit ist keine pauschale Einwilligung. Eine Verarbeitung kann zwar unter bestimmten Voraussetzungen auch auf einer anderen Rechtsgrundlage beruhen; diese muss aber eigenständig geprüft werden.

M12 · IRRTUM

Was sich aus Git-Aktivität erschließen lässt, ist eine Tatsache über die Person.

Git erfasst nur bestimmte dokumentierte Handlungen und lässt viele mögliche Ursachen für ein Muster offen. Gerade die Verknüpfung verschiedener Datenquellen kann heikle, aber dennoch unzuverlässige personenbezogene Zuschreibungen erzeugen.

M13 · IRRTUM

.gitignore schützt sensible Dateien zuverlässig vor Veröffentlichung.

Bereits versionierte Dateien bleiben von dieser Regel unberührt; frühere Versionsstände werden nicht nachträglich gesäubert. Auch versehentlich veröffentlichte Zugangsdaten erfordern eigene Gegenmaßnahmen.

M14 · IRRTUM

Mit der Veröffentlichung des Repositoriums sind alle Nutzungsrechte geklärt.

Textedition, Bildmaterial, Transkriptionen, Programmcode und Beiträge Dritter können unterschiedliche Rechte und Lizenzen betreffen. Öffentliche Lesbarkeit allein erteilt keine allgemeine Nachnutzungserlaubnis.

M15 · IRRTUM

Ein Git-Repositorium dokumentiert alle editorischen Entscheidungen.

Gespräche, nicht dokumentierte Alternativen, Vorarbeiten und die Motive hinter einer Änderung müssen im Repositorium nicht enthalten sein. Versionierung erfasst Resultate bestimmter Arbeitsschritte besser als deren gedankliche Entstehung.

M16 · IRRTUM

git blame zeigt, wer für eine Textstelle verantwortlich ist.

Der letzte Zeileneingriff kann eine formale Korrektur, automatische Umformatierung oder Übernahme fremder Arbeit sein. Die angezeigte Zuordnung ist technisch und hängt auch von den untersuchten Versionsständen ab.

M17 · IRRTUM

Git speichert jeden Zwischenstand automatisch.

Gewöhnlich werden erst durch einen Commit – einen explizit gespeicherten Versionsstand – dauerhaft nachvollziehbare Punkte in der Projektgeschichte gesetzt. Nicht gespeicherte Experimente und verworfene Zwischenstände können fehlen.

M18 · IRRTUM

Ein Commit entspricht genau einer editorischen Handlung.

Ein Commit kann mehrere Änderungen bündeln oder eine einzige Arbeit in viele kleine Einträge aufgeteilt werden. Die Granularität hängt von Gewohnheiten und Projektregeln ab.

M19 · IRRTUM

Die Reihenfolge der Commits ist der tatsächliche Entstehungsprozess.

Arbeiten können lange vor ihrer Speicherung erfolgen. Mit Funktionen wie Rebase (nachträgliches Neuordnen bzw. erneutes Anwenden von Commits) oder Squash (Zusammenfassen mehrerer Commits) lässt sich die sichtbare Versionsfolge verändern. Die dokumentierte Geschichte ist also teilweise gestaltbar.

M20 · IRRTUM

Commit-Zeitstempel belegen den Zeitpunkt einer editorischen Entscheidung.

Git unterscheidet Angaben zum ursprünglichen Autorzeitpunkt und zum Zeitpunkt der Übernahme in die Geschichte (Author Date und Committer Date). Beide sind Metadaten und nicht automatisch unabhängig beglaubigte Zeitangaben; die eigentliche Arbeit kann früher erfolgt sein.

M21 · IRRTUM

Der angegebene Commit-Autor ist zweifelsfrei die handelnde Person.

Diese Angaben sind grundsätzlich konfigurierbar. Bei gemeinsam erarbeiteten, importierten oder automatisiert gespeicherten Änderungen können die Angaben von der tatsächlichen Arbeitsteilung abweichen. Eine gesonderte Signatur- oder Identitätsprüfung ist eine andere Frage.

M22 · IRRTUM

Alles in einem Commit ist bewusst als editorische Aussage aufgenommen worden.

Ein Commit kann neben fachlichen Änderungen auch automatische Formatierung, Nebenfolgen eines Skripts oder versehentlich mitgespeicherte Anpassungen umfassen.

M23 · IRRTUM

Eine große Änderung in Git ist eine große editorische Veränderung.

Schon eine neue Einrückung oder das Umformatieren einer XML-Datei kann zahlreiche Zeilen als verändert erscheinen lassen, obwohl die editorische Bedeutung nahezu gleich bleibt.

M24 · IRRTUM

Ein Revert macht die frühere Entscheidung bedeutungslos.

Dass eine Lesart oder Auszeichnung zunächst gewählt und später verworfen wurde, kann für die Geschichte der Edition besonders aufschlussreich sein. Das technische Rückgängigmachen erklärt die Gründe nicht.

M25 · IRRTUM

Git verhindert Konflikte und Datenverluste bei gemeinsamer Arbeit.

Beim Merge – der Zusammenführung zweier Entwicklungsstände – können Konflikte entstehen. Sie müssen sachgerecht gelöst werden. Auch unerkannte Fehler, falsche Bedienung oder ungesicherte Daten bleiben möglich.

M26 · IRRTUM

Wenn Git beim Zusammenführen keinen Konflikt meldet, passen die Änderungen fachlich zusammen.

Zwei Änderungen an unterschiedlichen Stellen können sich inhaltlich widersprechen, obwohl Git sie technisch problemlos zusammenfügt – etwa wenn Auszeichnungsrichtlinien an zwei Stellen unterschiedlich umgesetzt werden.

M27 · IRRTUM

Branches sind automatisch unterschiedliche wissenschaftliche Textfassungen.

Branches werden ebenso für technische Reparaturen, Experimente, Arbeitsorganisation oder kurzfristige Tests benutzt. Ihre wissenschaftliche Bedeutung ergibt sich aus der konkreten Projektpraxis.

M28 · IRRTUM

Git organisiert Teamarbeit von allein.

Teams benötigen Absprachen über Zuständigkeiten, Speicherhäufigkeit, Benennungen, Konfliktlösung, Qualitätsprüfung und Entscheidungsbefugnisse.

M29 · IRRTUM

Ein Pull Request ist automatisch wissenschaftliches Peer Review.

Die Prüfung kann sich auf technische Funktionsfähigkeit beschränken oder nur formell erfolgen. Inhaltliche Begutachtung benötigt eigene Kriterien und Verantwortlichkeiten. Das Verfahren ist zudem eine Plattformfunktion, keine zwingende Eigenschaft von Git.

M30 · IRRTUM

Gültiges TEI-XML ist auch editorisch richtig.

Formale Gültigkeit prüft nicht automatisch die historische Richtigkeit einer Lesart, die Begründung einer Normalisierung oder die Angemessenheit einer Auszeichnung.

M31 · IRRTUM

Textbasierte XML-Dateien ergeben automatisch gut lesbare Git-Vergleiche.

Formatierungsänderungen, lange XML-Zeilen oder unterschiedlich angeordnete Elemente erschweren die Interpretation. Git vergleicht zunächst Zeichen bzw. Zeilen und versteht die XML-Struktur oder editorische Semantik nicht von selbst.

M32 · IRRTUM

Ein Git-Commit ist ein fälschungssicherer historischer Beleg.

Prüfsummen sichern eine technische Beziehung zwischen Daten und Kennwerten; sie beglaubigen nicht die Wahrheit der eingetragenen Autor-, Zeit- oder Entscheidungsangaben. Versionen können neu aufgebaut werden, und die Echtheit einer Quelle hängt von ihrem Überlieferungskontext ab.

M33 · IRRTUM

Ein gespeicherter Git-Stand macht eine Edition vollständig reproduzierbar.

Zusätzlich können Programme, externe Daten, Schriftarten, Betriebssysteme, Konfigurationen, Datenbanken und nicht versionierte Ressourcen nötig sein. Versionierte Quelldaten sind ein wichtiger Baustein, aber nicht die gesamte Ausführungsumgebung.

M34 · IRRTUM

Git ist ein Backup.

Eine einzige lokale Kopie schützt nicht vor Defekten oder Verlust des Geräts. Auch synchronisierte Kopien können Fehler übernehmen; nicht erfasste Dateien und externe Speicherbestände fehlen möglicherweise. Backups benötigen getrennte, getestete Wiederherstellungswege.

M35 · IRRTUM

Ein Repositorium auf GitHub oder GitLab ist langfristig archiviert.

Hosting-Verträge, Konten, Speicherregeln, Verfügbarkeit externer Dateien und die langfristige Erschließung können sich ändern. Archivierung verlangt organisatorische, technische und dokumentarische Maßnahmen über das bloße Hosting hinaus.

M36 · IRRTUM

Ein DOI macht ein Repositorium unveränderlich.

Ein DOI identifiziert ein digitales Objekt oder einen Veröffentlichungsstand nach den Regeln des jeweiligen Dienstes. Ob die zugehörigen Daten tatsächlich als unveränderliche Version aufbewahrt werden, hängt von dessen Verfahren ab. Ein laufend veränderliches Repositorium und eine archivierte Veröffentlichungsversion müssen unterschieden werden.

M37 · IRRTUM

Ein Tag garantiert eine verlässliche Publikationsfassung.

Tags kennzeichnen technisch einen Versionsstand, begründen aber keinen Qualitätssicherungs- oder Archivierungsprozess. Unter bestimmten Bedingungen können sie auch geändert oder gelöscht werden.

M38 · IRRTUM

Open Source bei Git bedeutet Unabhängigkeit vom Plattformanbieter.

Zusätzliche Plattformfunktionen – etwa Tickets für Aufgaben, Diskussionen, Zugriffsverwaltung oder automatisierte Abläufe – lassen sich nicht immer vollständig oder verlustfrei auf einen anderen Dienst übertragen.

M39 · IRRTUM

Das Repositorium enthält die gesamte Projektdokumentation.

Entscheidungen aus Besprechungen, E-Mails, externen Ticketsystemen und Datenbanken sowie nicht übernommene Arbeitsmaterialien fehlen möglicherweise. Selbst eine vollständige Git-Geschichte ist daher nicht notwendig eine vollständige Projektüberlieferung.

M40 · IRRTUM

Git eignet sich für alle Medien und Dateiformate gleich gut.

Reine Textdateien lassen sich oft besonders gut vergleichen. Große oder binäre Dateien beanspruchen unter Umständen mehr Speicher und ermöglichen weniger aussagekräftige Vergleiche. Git LFS („Large File Storage“) ist eine Erweiterung, bei der große Inhalte separat gespeichert und im Git-Repositorium über Verweise eingebunden werden; dies schafft eigene Anforderungen an Sicherung und Archivierung.

M41 · IRRTUM

Eine Git-Versionsgeschichte lässt sich ohne Kontext verstehen.

Editionsrichtlinien, Rollenverteilungen, verwendete Werkzeuge, Arbeitskonventionen und zeitliche Projektphasen geben Änderungen erst ihren historischen Sinn. Eine Umstellung der TEI-Auszeichnung kann fachlich, organisatorisch oder technisch begründet sein.

M42 · IRRTUM

Aus einem Git-Unterschied lässt sich die editorische Entscheidung automatisch ablesen.

Dieselbe sichtbare Änderung kann durch eine neue Interpretationsentscheidung, ein geändertes Auszeichnungsschema, eine automatische Umwandlung oder die Korrektur eines Fehlers entstanden sein.

M43 · IRRTUM

Viele Commits bedeuten hohe Produktivität.

Menschen und Projekte speichern Änderungen unterschiedlich häufig; automatische Prozesse können zusätzliche Einträge erzeugen. Zahl und Qualität wissenschaftlicher Entscheidungen sind aus der bloßen Commitmenge nicht abzuleiten.

M44 · IRRTUM

Wer keine Commits erzeugt, arbeitet nicht.

Literaturrecherche, Quellenvergleich, Diskussion, Korrekturlesen, konzeptionelle Arbeit und lokale Bearbeitungen ohne anschließenden Commit können unsichtbar bleiben.

M45 · IRRTUM

Commit-Zeiten sind Arbeitszeiten.

Commits können gebündelt, verzögert, automatisiert oder zu anderen Zeitpunkten als der eigentlichen Arbeit erstellt werden. Sie bilden weder die gesamte Tätigkeit noch zwingend deren zeitliche Dauer ab.

M46 · IRRTUM

Viele bearbeitete Dateien zeigen einen hohen Editionsfortschritt.

Ein einzelner schwieriger Quellenbefund kann erheblichen Aufwand erfordern, während automatisierte Umformungen tausende Dateien betreffen. Fachlicher Fortschritt braucht projektspezifische Kriterien.

M47 · IRRTUM

Git-Daten verschiedener Editionsprojekte sind direkt vergleichbar.

Projekte unterscheiden sich unter anderem nach Dateigröße, Aufgaben, Mitarbeitendenzahl, Arbeitsabläufen, Commit-Praxis, Automatisierungsgrad und Repositoriumstyp. Ein Arbeitsrepositorium dokumentiert laufende Bearbeitung meist anders als ein Publikationsrepositorium, das vor allem bereitgestellte Ausgaben enthält. Vergleiche brauchen daher Kontext und methodische Normalisierung.

F01 · FAKT

Git lässt sich direkt aus Oxygen XML Editor bedienen

Das Git Client Add-on für Oxygen XML Editor stellt häufige Git-Funktionen in der gewohnten XML-Arbeitsumgebung bereit: Unterschiede prüfen, Commits anlegen, Änderungen übertragen und empfangen, Entwicklungszweige verwalten und Konflikte behandeln. Für diese Standardaufgaben muss man nicht zwingend Befehle eintippen. Wer mit TEI/XML arbeitet, kann Versionsverwaltung und Textbearbeitung in einer Anwendung verbinden. Das senkt die Einstiegshürde. Die Oberfläche vereinfacht die Bedienung, ersetzt aber kein Grundverständnis der Versionsverwaltung. Ein Konflikt bei der Zusammenführung muss weiterhin inhaltlich beurteilt werden.

F02 · FAKT

Beim Hochladen von Änderungen können automatische Prüfungen starten

Ein Push (Übertragung lokaler Commits an ein entferntes Repo) kann auf Hosting-Plattformen automatisierte Arbeitsabläufe auslösen. Diese werden häufig CI/CD-Pipelines genannt: definierte Prüfschritte oder Verarbeitungsschritte, die nach einem Ereignis ausgeführt werden. TEI-Dateien können automatisch auf wohlgeformtes XML, die Einhaltung eines Relax-NG-Schemas oder zusätzliche Schematron-Regeln geprüft werden. Ein fehlerhafter Datenstand fällt frühzeitig auf. Git selbst führt durch einen Push nicht automatisch eine Schema-Prüfung aus. Dafür braucht es eingerichtete Programme und eine Ausführungsumgebung, etwa GitHub Actions oder GitLab CI/CD. Schema-Gültigkeit ist keine Garantie editorischer Richtigkeit.

F03 · FAKT

Aus einem Commit kann automatisch eine neue Editionsvorschau entstehen

Die Automatisierung kann über Prüfungen hinausgehen: Aus TEI/XML lassen sich bei neuen Commits mittels XSLT und weiterer Werkzeuge HTML-Seiten, Register, Suchindizes oder Publikationsdateien erzeugen. Die veröffentlichte Ansicht kann wiederholt und nachvollziehbar aus einem festgelegten Datenstand erstellt werden; Änderungen werden zeitnah sichtbar. Das ist ein zusätzlich eingerichteter Publikationsprozess, keine eingebaute Eigenschaft von Git. Für Wiederholbarkeit müssen auch Softwareversionen, externe Daten und Einstellungen dokumentiert werden.

F04 · FAKT

Ein Repository kann die Übernahme ungeprüfter Änderungen erschweren

Hosting-Plattformen unterstützen Schutzregeln für wichtige Branches (Entwicklungszweige). Man kann beispielsweise verlangen, dass automatische Tests erfolgreich sind oder eine zweite Person die Änderung geprüft hat, bevor sie in die gemeinsame Hauptfassung gelangt. Ein Team kann technische Mindeststandards – etwa erfolgreiche XML-Validierung – verbindlich in den Arbeitsablauf integrieren. Solche Regeln gelten nur dort, wo sie eingerichtet und nicht umgangen werden können. Eine Freigabe ist nicht automatisch eine wissenschaftliche Begutachtung.

F05 · FAKT

Schon vor dem Anlegen eines Commits können Programme gestartet werden

Git Hooks sind Skripte, die auf bestimmte Git-Ereignisse reagieren. Ein pre-commit hook wird beispielsweise vor dem Anlegen eines neuen Commits ausgeführt. Auf einem lokalen Rechner kann automatisch eine XML-Prüfung, eine Formatkontrolle oder die Suche nach versehentlich aufgenommenen Dateien beginnen. Lokale Hooks müssen eingerichtet werden und lassen sich umgehen. Sie ersetzen keine zentralen Regeln für gemeinsam veröffentlichte Daten.

F06 · FAKT

Mehrere Entwicklungsstände können gleichzeitig geöffnet bleiben

Mit Git Worktree lassen sich mehrere Arbeitsverzeichnisse anlegen, die auf unterschiedliche Branches desselben Repositories zeigen. Man kann etwa die veröffentlichte Edition in einem Verzeichnis betrachten und eine neue TEI-Auszeichnungsstrategie parallel in einem anderen erproben. Die Trennung betrifft Arbeitsverzeichnisse, nicht automatisch unabhängige Datenbestände, Zugriffskontrollen oder Archivkopien.

F07 · FAKT

Ein Commit enthält weit mehr als eine Commit-Nachricht

Ein Commit enthält eine eindeutige Kennung, einen Verweis auf den gespeicherten Dateibestand, Verweise auf Vorgänger-Commits, Angaben zu Autor und Ersteller, Zeitangaben und eine Nachricht. Gegebenenfalls kommt eine digitale Signatur hinzu. Die Unterschiede zwischen zwei Ständen lassen sich daraus und aus den zugehörigen Dateibeständen berechnen. Es lassen sich technische Abfolgen, Zuschreibungen und Bearbeitungskontexte untersuchen – auch wenn die erklärende Nachricht knapp ist. Die Metadaten beweisen nicht ohne Weiteres die Identität der handelnden Person, die Arbeitsdauer oder die Gründe einer Entscheidung.

F08 · FAKT

Autor und technische Erstellerin eines Commits können verschieden sein

Git unterscheidet Author (wem die Änderung zugeschrieben wird) und Committer (wer den Commit in seiner vorliegenden Form erzeugt hat). Das hilft, eingereichte Beiträge, übernommene Änderungen oder die Übertragung älterer Projektbestände differenzierter zu verstehen. Beide Angaben sind konfigurierbare Metadaten, nicht automatisch überprüfte Identitätsnachweise.

F09 · FAKT

Die Geschichte einzelner Textzeilen lässt sich verfolgen

git blame kann anzeigen, welcher Commit eine Zeile zuletzt geändert hat; bestimmte Optionen berücksichtigen auch verschobene oder kopierte Zeilen. Die Entstehung einer Lesart oder einer TEI-Auszeichnung lässt sich bis zu einem früheren Bearbeitungsstand zurückverfolgen. Bei Umformatierungen kann die Zuordnung irreführend werden. Der zuletzt dokumentierte Eingriff sagt nicht automatisch etwas über wissenschaftliche Verantwortung aus.

F10 · FAKT

Man kann die Geschichte nach bestimmten Textänderungen durchsuchen

Git kann nach Commits suchen, bei denen eine Zeichenfolge hinzugefügt oder entfernt wurde. Die dafür bekannte Suchfunktion heißt Pickaxe. So lässt sich beispielsweise untersuchen, wann bestimmte TEI-Elemente, Namensformen oder Referenzen häufiger beziehungsweise seltener wurden. Die Suche ist zeichenfolgenorientiert und kein vollwertiger semantischer XML-Vergleich. Sie deckt nicht automatisch jede inhaltlich relevante Änderung auf.

F11 · FAKT

Beliebige historische Projektstände lassen sich vergleichen

Ein Diff ist die Darstellung von Unterschieden zwischen zwei Bearbeitungsständen. Git kann auch Stände vergleichen, zwischen denen Jahre und sehr viele Commits liegen. Sichtbar werden langfristige Umstrukturierungen, neue Dokumentgruppen und die Entwicklung einer Editionsfassung. Der Vergleich zeigt Unterschiede zwischen den gewählten Zuständen, nicht alle dazwischenliegenden Überlegungen oder Entscheidungen.

F12 · FAKT

Git kann Umbenennungen von Dateien erkennen

Beim Vergleich von Bearbeitungsständen kann Git anhand inhaltlicher Ähnlichkeit darauf schließen, dass eine Datei verschoben oder umbenannt wurde. Das hilft, Texte durch Änderungen von Ordnerstruktur, Signaturen oder Dateinamen hinweg zu verfolgen. Eine Umbenennung wird in Git nicht grundsätzlich als eigenes Ereignis gespeichert; die Erkennung ist abhängig von Vergleichsoptionen und Ähnlichkeitsgrenzen.

F13 · FAKT

Der Ursprung eines Fehlers lässt sich durch systematisches Halbieren finden

git bisect prüft schrittweise ältere Commits, um den Übergang von einem funktionierenden zu einem fehlerhaften Zustand einzugrenzen. Hilfreich, wenn eine XSLT-Transformation, ein Register oder die Generierung einer Editionswebsite irgendwann fehlerhaft geworden ist. Man benötigt einen sinnvollen Test, mit dem sich für untersuchte Versionen feststellen lässt, ob das Problem schon auftritt.

F14 · FAKT

Ältere Commits lassen sich nachträglich kommentieren, ohne sie umzuschreiben

Git Notes sind zusätzliche Anmerkungen, die einem Commit zugeordnet werden können, ohne dessen ursprünglichen Inhalt zu verändern. Eine spätere quellenkritische Einordnung kann von der ursprünglich überlieferten Bearbeitung getrennt bleiben. Solche Notizen werden nicht in allen Programmen angezeigt und nicht automatisch bei jeder Synchronisierung übertragen.

F15 · FAKT

Manche vermeintlich verlorenen Bearbeitungsstände lassen sich wiederfinden

Das lokale Reflog zeichnet auf, wohin Git-Verweise zuletzt gezeigt haben. Damit lassen sich unter Umständen auch nach einer versehentlichen Rücksetzung ältere Commits wiederfinden. Nützlich bei Bedienfehlern oder versehentlich verworfenen Entwicklungszweigen. Die Aufzeichnungen sind lokal, können ablaufen und bereinigt werden. Das Reflog ist kein Backup.

F16 · FAKT

Die gesamte Entwicklung lässt sich als verzweigtes Diagramm ansehen

Grafische Git-Programme stellen Commits als Punkte und ihre technischen Beziehungen als Verbindungslinien dar. So sieht man Entwicklungszweige und deren Zusammenführung (Merge) auf einen Blick. Parallele Bearbeitung, Integrationsphasen und längere Entwicklungslinien werden leichter verständlich. Das Diagramm zeigt Beziehungen zwischen gespeicherten Ständen, nicht alle Kommunikations- oder Entscheidungsprozesse.

F17 · FAKT

Ein Git-Projekt kann als animierte Landschaft gezeigt werden

Das Visualisierungsprogramm Gource zeigt die Entwicklung eines Repositories als Animation: Verzeichnisse, Dateien und protokollierte Beiträge erscheinen als bewegte Elemente. Sehr geeignet, um Entstehung, Wachstum und Umstrukturierung eines Projekts in einem Vortrag zu zeigen. Die Animation basiert auf den sichtbaren Git-Daten. Sie zeigt nicht den gesamten Arbeitsaufwand und kann Muster ästhetisch eindrucksvoller erscheinen lassen, als sie methodisch belastbar sind.

F18 · FAKT

Aus Git-Metadaten lassen sich Zeitreihen und Aktivitätsdiagramme erzeugen

Commit-Zeitpunkte, geänderte Dateien und dokumentierte Bearbeitungshäufigkeiten lassen sich statistisch aufbereiten und visualisieren. Damit lassen sich Arbeitsphasen, technische Umstrukturierungen und unterschiedliche Projektpraktiken untersuchen. Die Zahl der Commits ist weder ein verlässlicher Arbeitszeitnachweis noch ein objektives Produktivitätsmaß. Personenbezogene Analysen werfen zusätzliche Datenschutzfragen auf.

F19 · FAKT

Unterschiede lassen sich auch innerhalb einzelner Zeilen hervorheben

Git kann statt ganzer geänderter Zeilen auch kleinere Textelemente als Unterschiede hervorheben; diese Darstellung heißt Word Diff. Besonders hilfreich bei langen XML-Zeilen, in denen nur ein Wort oder ein kleiner Abschnitt geändert wurde. Ein Wortvergleich ist noch kein Vergleich der semantischen Bedeutung von TEI-Elementen.

F20 · FAKT

Vergleiche lassen sich für unterschiedliche Dateitypen anpassen

Mit einer Git-Konfigurationsdatei namens .gitattributes können Dateitypen unterschiedlich behandelt und eigene Vergleichsprogramme eingebunden werden. XML-Dateien lassen sich für Vergleiche gezielt aufbereiten, um rein technische Formatierungsänderungen besser von inhaltlich interessanten Eingriffen zu trennen. Ein projektspezifischer Vergleich benötigt zusätzliche Einrichtung und ist nicht automatisch wissenschaftlich interpretierend.

F21 · FAKT

Git funktioniert auch vollständig lokal und offline

Dateien versionieren, Commits anlegen, Unterschiede vergleichen und Branches verwalten funktioniert ohne Hosting-Plattform und ohne Internetzugang. Auch Einzelpersonen können die eigene Transkriptions- und Auszeichnungspraxis nachvollziehbar gestalten. Ohne eine zusätzliche, unabhängig aufbewahrte Kopie ist die lokale Versionsgeschichte nicht vor Geräteverlust geschützt.

F22 · FAKT

Eine Git-Versionsgeschichte lässt sich in einer Datei bündeln

Ein Git Bundle ist eine transportable Datei, die ausgewählte Git-Objekte und Verweise eines Repositories enthält. Sie kann ohne einen laufenden Git-Server weitergegeben und wieder eingelesen werden. Geeignet für Offline-Transfers, technische Projektübergaben und bestimmte Sicherungsszenarien. Ein Bundle enthält nicht automatisch ungespeicherte Arbeitsdateien, sämtliche lokalen Einstellungen oder extern gespeicherte Inhalte wie Git-LFS-Objekte.

F23 · FAKT

Aus einem historischen Stand lässt sich ein reines Datenarchiv erzeugen

Mit git archive kann man die Dateien eines bestimmten Commits als ZIP- oder TAR-Archiv exportieren – ohne die ganze Git-Historie mitzugeben. Damit kann ein klar definierter Publikationsstand veröffentlicht werden, während die interne Arbeitsgeschichte im Projekt bleibt. Ein Dateiexport ohne Versionsgeschichte dokumentiert selbst nicht, wie die Daten entstanden sind. Auch externe Abhängigkeiten müssen gesondert berücksichtigt werden.

F24 · FAKT

Veröffentlichungsstände lassen sich mit Namen markieren

Ein Tag ist eine benannte Markierung eines bestimmten Commits – ähnlich einem Lesezeichen in der Geschichte des Projekts. edition-1.0 könnte beispielsweise den Stand einer veröffentlichten Edition bezeichnen. Bestimmte Fassungen werden wiederauffindbar und lassen sich leichter zitieren oder exportieren. Ein Tag ist noch keine Publikation, kein DOI und kein Archivierungsnachweis. Solche Markierungen können technisch verändert oder gelöscht werden.

F25 · FAKT

Commits und Tags können digital signiert werden

Bestimmte Git-Objekte lassen sich kryptografisch signieren. Unter geeigneten Bedingungen lässt sich dann prüfen, ob der signierte Stand zur verwendeten Signatur und dem entsprechenden Schlüssel passt. Signaturen können technische Nachweise zu einer freigegebenen Fassung unterstützen. Sie beweisen weder die inhaltliche Richtigkeit der Edition noch allein die reale Identität oder wissenschaftliche Verantwortung einer Person.

F26 · FAKT

Auf einzelne Zeilen eines bestimmten historischen Stands lässt sich verlinken

Plattformen wie GitHub können dauerhafter angelegte Links auf Dateien in einem ganz bestimmten Commit bereitstellen, gegebenenfalls bis zur einzelnen Zeile. In einer Forschungsdiskussion lässt sich auf den genauen damaligen Stand einer TEI-Datei verweisen statt auf die stets veränderliche aktuelle Fassung. Die dauerhafte Erreichbarkeit hängt von der Aufbewahrung und Zugänglichkeit ab. Zeilennummern sind keine editorisch stabilen Textanker.

F27 · FAKT

Die Entwicklung des Textes und seiner Werkzeuge kann gemeinsam dokumentiert werden

Im selben Repository können neben TEI-Dateien auch Schema-Dateien, XSLT-Transformationen, Skripte, ODD-Spezifikationen und Konfigurationen versioniert werden. Ein späterer Vergleich kann zeigen, ob Änderungen an der Datenauszeichnung zusammen mit Änderungen der Editionsrichtlinien oder technischen Verarbeitung vorgenommen wurden. Die gemeinsame Aufbewahrung erzeugt noch keine automatische Erklärung kausaler Zusammenhänge. Externe Systeme müssen zusätzlich dokumentiert werden.

F28 · FAKT

Alternative Editionsrichtlinien lassen sich ausprobieren, ohne den Hauptstand zu überschreiben

Ein Branch ist ein Entwicklungszweig, in dem man Änderungen unabhängig vom gemeinsam genutzten Hauptstand weiterführen kann. Später lassen sich die Arbeiten gegebenenfalls wieder zusammenführen (Merge). Alternative TEI-Auszeichnungsmodelle, Normalisierungen oder Transformationswege können erprobt und verglichen werden. Die spätere Zusammenführung kann technische Konflikte erzeugen. Auch konfliktfreie Änderungen können editorisch unvereinbar sein.

F29 · FAKT

Ein Commit muss nicht alle Änderungen einer Datei enthalten

Git kann einzelne Änderungsabschnitte für einen Commit auswählen. Das wird oft Partial Staging genannt – also das gezielte Vormerken nur eines Teils der Änderungen. Eine orthografische Korrektur und eine Änderung der XML-Auszeichnung können getrennt dokumentiert werden, obwohl sie dieselbe Datei betreffen. Ausgewählte Teilschritte können vorübergehend ungültiges XML ergeben. Deshalb sollte der zu speichernde Gesamtzustand geprüft werden.

F30 · FAKT

Unfertige Arbeit lässt sich zwischenspeichern

Mit Git Stash können noch nicht in einem Commit festgehaltene Änderungen zeitweilig beiseitegelegt und später wieder eingespielt werden. Man kann kurzfristig eine dringende Korrektur vornehmen und anschließend zu einer längeren, noch unfertigen Bearbeitung zurückkehren. Diese Zwischenspeicherung ist keine langfristige wissenschaftliche Dokumentation und keine sichere Backup-Strategie.

F31 · FAKT

Ein Clone enthält häufig auch die Vorgeschichte

Ein Clone ist eine lokale Kopie eines Repositories. Im Normalfall werden dabei nicht bloß die aktuellen Dateien, sondern auch die erreichbare Git-Geschichte übertragen. Ein Repo kann unabhängig von der Serververbindung untersucht und weiterbearbeitet werden. Es gibt auch eingeschränkte Kopierverfahren. Für Datenschutz und Veröffentlichung entscheidend ist, dass historische Metadaten und frühere Dateifassungen mitverbreitet werden können.

F32 · FAKT

Unterschiedliche Namensangaben können für Statistiken zusammengeführt werden

Mit einer Datei namens .mailmap lassen sich unterschiedliche Namen oder E-Mail-Adressen derselben Person für bestimmte Anzeigen und Auswertungen vereinheitlichen, ohne die alten Commits umzuschreiben. Besonders wichtig bei langfristigen Projekten, in denen Beteiligte unter mehreren Adressen oder Schreibweisen auftauchen. Die Zuordnung ist interpretativ und muss begründet werden; gleiche Namen bedeuten nicht zwingend gleiche Personen.

F33 · FAKT

Manche Dateien lassen sich bewusst anders vergleichen

Über .gitattributes kann Git für bestimmte Dateien beispielsweise auf die gewöhnliche zeilenweise Vergleichsdarstellung verzichten oder externe Vergleichswerkzeuge einsetzen. Für erzeugte, binäre oder strukturell besondere Dateien lässt sich eine passende Darstellung wählen. Eine andersartige Darstellung verbirgt den Inhalt nicht vor berechtigten Nutzenden; sie ist keine Datenschutz- oder Zugriffsschutzfunktion.

F34 · FAKT

Große Begleitdateien können außerhalb der gewöhnlichen Git-Objekte liegen

Git Large File Storage (Git LFS) ergänzt Git um ein Verfahren für große Dateien: Im Git-Repository liegen Verweise, die eigentlichen Inhalte werden in einer zusätzlichen Speicherstruktur abgelegt. Hilfreich für hochauflösende Faksimiles, Audio-/Videodateien oder große Binärbestände. Für vollständige Übergaben und Langzeitarchivierung müssen die externen Dateiobjekte ausdrücklich mitgesichert werden.

F35 · FAKT

Arbeitsgeschichte und veröffentlichte Daten können getrennt werden

Man kann intern ein umfassendes Arbeitsrepository führen und für die Öffentlichkeit nur geprüfte Dateien oder einen bereinigten Datenexport bereitstellen – auch automatisiert. Das erlaubt Transparenz über publizierte Forschungsdaten, ohne sämtliche internen Diskussions- und Arbeitsmetadaten offenlegen zu müssen. Die Trennung muss konzeptionell und technisch zuverlässig eingerichtet werden; eine bloße Sichtbarkeitseinstellung bereinigt keine bereits veröffentlichte Versionsgeschichte.

F36 · FAKT

Ein Git-Projekt ist nicht an eine Hosting-Plattform gebunden

Die eigentliche Git-Geschichte lässt sich grundsätzlich auf einen anderen Dienst oder einen institutionellen Server übertragen. Das schafft Spielraum für Migration, institutionelle Kontrolle und langfristige Verfügbarkeit von Forschungsdaten. Plattformgebundene Zusatzinformationen wie Tickets (Issues), Diskussionsverläufe, Freigaben oder Zugriffsregeln sind nicht automatisch Teil der portablen Git-Geschichte.

Antipatterns erkennen und besser handeln

Eine zunächst plausible Lösung kann unter bestimmten Bedingungen Nachteile erzeugen. Die Geltungsgrenze nennt Fälle, in denen die Praxis dennoch sinnvoll ist.

A1

Die Version im Dateinamen

Muster: Jede Fassung wird als separate Datei abgelegt; Versionsnummern, Datums- und Personenzusätze stehen im Namen.

Besser: Versionskontrolle bei stabilen Dateinamen; definierte Exporte mit eindeutigen Versionskennungen.

Geltungsgrenze: Versionsangaben in unveränderlichen Publikationsexporten sind legitim; problematisch ist die Verwendung solcher Dateikopien als ausschließliche Arbeitsversionierung.

A2

Der Kopienzoo

Muster: Unabhängig bearbeitete Ordnerkopien liegen auf Rechnern, Datenträgern und Laufwerken und werden informell ausgetauscht.

Besser: Geregelte Synchronisation mit Versionskontrolle sowie davon unabhängige Backups.

Geltungsgrenze: Kontrollierte Sicherungskopien, Archiv-Snapshots und ausdrücklich unveränderliche Exporte sind sinnvoll.

A3

Die Geschichte ohne Vergangenheit

Muster: Ältere Bearbeitungsstände werden überschrieben oder nach der Publikation verworfen.

Besser: Versionierung und eine bewusste, rechtlich geprüfte Bewahrungsstrategie für wissenschaftlich relevante Prozessdokumentation.

Geltungsgrenze: Nicht jede flüchtige Zwischenfassung ist archivierungswürdig. Ein Publikationsrepository muss nicht die gesamte interne Arbeitshistorie enthalten.

D3

Git ist unser Backup

Muster: Das vorhandene Git-Repository ersetzt alle unabhängigen Sicherungsverfahren.

Besser: Dokumentierte, getestete Backup- und Wiederherstellungsstrategie, in der Git eine Komponente sein kann.

Geltungsgrenze: Git kann einen wesentlichen Teil einer robusten Sicherungsstrategie bilden.

D4

Das herrenlose Repository

Muster: Administration, Rechteklärung und Entscheidungen bleiben informell an einzelnen Personen hängen, ohne geregelte Vertretung oder Übergabe.

Besser: Dokumentierte institutionelle, fachliche, technische und datenschutzbezogene Verantwortlichkeiten sowie Vertretungs- und Übergaberegeln.

Geltungsgrenze: Einzelpersonen dürfen zuständig sein; die Verantwortung muss jedoch übertragbar und angemessen abgesichert sein.

B1

Metadaten auf Wanderschaft

Muster: Sie stehen ausschließlich in Commitnachrichten, Tags, Branches, Issues, Dateinamen oder Ordnern statt in geeigneten strukturierten Metadaten.

Besser: TEI-Header (wo passend, etwa revisionDesc), fachliche Metadatendateien oder andere geeignete strukturierte Modelle; ergänzende Git-Verweise.

Geltungsgrenze: Provisorische oder redundante Git-Informationen sind legitim. Zum Antipattern wird die Praxis, wenn dauerhaft benötigte fachliche Angaben exklusiv dort verbleiben.

B2

Der Branch als Editionsmodell

Muster: Ihre fachliche Identität und Beziehung werden ausschließlich durch Branches repräsentiert.

Besser: Eigenständige fachliche Repräsentationen und Beziehungen, gegebenenfalls Transformationen; Branches zum Entwickeln, definierte Releases für zitierfähige Stände.

Geltungsgrenze: Auch langfristig gepflegte Fassungsbranches sind vertretbar, solange ihre wissenschaftliche Bedeutung ausdrücklich dokumentiert wird.

B3

Die versteckte Editionsentscheidung

Muster: Die einzige Begründung steht in Commitnachrichten, Issues oder Pull-Request-Diskussionen.

Besser: Publikationsrelevante Begründungen in Apparat, Kommentar oder Editionsdokumentation überführen; Git-Diskussionen ergänzend verknüpfen.

Geltungsgrenze: Nicht jede Arbeitsdiskussion muss publiziert werden; relevant ist die notwendige Begründung der veröffentlichten Edition.

B4

Der verwaiste Editionsstatus

Muster: Ein operativer Status (geschlossenes Issue, gemergter Branch, done im Board) gilt automatisch als fachliche Freigabe.

Besser: Expliziter, nachvollziehbarer Freigabeprozess mit definierten Prüfbedingungen und dokumentiertem Ergebnis.

Geltungsgrenze: Ein Projektboard kann die Freigabe abbilden, wenn der Status tatsächlich an überprüfbare Kriterien und Verantwortlichkeiten gebunden ist.

C5

Der ungeprüfte Publikationsstand

Muster: Neue Arbeitsstände werden ohne definierte Qualitätsprüfung direkt als Publikationsstand bereitgestellt.

Besser: Validierung, projektspezifische Tests und explizite Publikationsfreigabe.

Geltungsgrenze: Öffentlich sichtbare experimentelle Arbeitsstände dürfen unfertig sein, wenn ihr Status eindeutig ist; direkte Commits auf main sind nicht selbst das Problem.

G1

Die stillschweigende Git-Vereinbarung

Muster: Mit dem Repository-Zugang gelten Rechte, Pflichten und künftige Nutzungen der Arbeitsdaten stillschweigend als geklärt.

Besser: Verbindliche und verständliche Repository-Regeln; ergänzende rechtlich notwendige Vereinbarungen und Informationen.

Geltungsgrenze: Bestehende institutionelle Regelungen können ausreichen; nicht jede Frage erfordert einen separaten individuellen Vertrag.

D1

Der ewige Zugang

Muster: Berechtigungen werden beim Einstieg vergeben, aber bei Aufgabenwechsel oder Ausscheiden nicht überprüft oder entzogen.

Besser: Rollenbezogene Vergabe minimal notwendiger Rechte, regelmäßige Rezertifizierung und verbindliches On- und Offboarding, auch für Service-Accounts.

Geltungsgrenze: Eine fortdauernde Mitarbeit kann eine erneute, ausdrücklich begründete Berechtigung rechtfertigen. Entzug verhindert nicht die Nutzung schon angefertigter lokaler Kopien.

G4

Die ungeklärte Rechtekette

Muster: Die Möglichkeit, Beiträge in ein gemeinsames Repository zu committen, gilt als hinreichende Berechtigung für alle späteren Nutzungen.

Besser: Schutzstatus, Rechtsinhaber und notwendige Nutzungsbefugnisse für unterschiedliche Beitragsarten frühzeitig klären und dokumentieren.

Geltungsgrenze: Nicht jeder Beitrag ist urheberrechtlich geschützt; Rechte können bereits durch Verträge oder andere Regelungen geklärt sein. Anwendbares Recht ist maßgeblich.

G5

Die Lizenz per Knopfdruck

Muster: Eine einzige Lizenzdatei wird undifferenziert für TEI, wissenschaftliche Texte, Software, Übersetzungen und Fremdmaterial eingesetzt.

Besser: Klare, ressourcenspezifische Lizenz- und Rechteinformationen; einheitliche Lizenz nur bei tatsächlich einheitlicher Rechtslage.

Geltungsgrenze: Eine einheitliche Lizenz ist sinnvoll, wenn sie tatsächlich alle betroffenen Ressourcen wirksam abdeckt.

G2

Die pauschale Einwilligung

Muster: Eine umfassende Anfangserklärung wird als Freigabe aller gegenwärtigen und zukünftigen Auswertungen und Veröffentlichungen behandelt.

Besser: Verarbeitungszwecke, geeignete Rechtsgrundlagen, Informationen und Schutzmaßnahmen je Nutzung klären; wo passend wirksame spezifische Einwilligung.

Geltungsgrenze: Eine freiwillige, informierte und hinreichend bestimmte Einwilligung kann in geeigneten Konstellationen legitim sein; sie ist nicht die einzige mögliche Rechtsgrundlage.

E1

Der Commit als Leistungsnachweis

Muster: Commitanzahl, geänderte Zeilen und Frequenz werden unmittelbar als Leistung oder Produktivität gedeutet.

Besser: Fachlich begründete, kontextbezogene Kriterien; qualitative Bewertung; zusätzliche Berücksichtigung nicht dokumentierter Arbeit.

Geltungsgrenze: Als deskriptive Projektmetrik sind solche Zahlen nützlich; ob ihre personenbezogene Verarbeitung erlaubt ist, ist zusätzlich gesondert zu klären (G3).

E2

Das Git als allwissender Zeuge

Muster: Die Git-Historie wird als vollständige Aufzeichnung wissenschaftlicher Arbeit und Entscheidungsfindung behandelt.

Besser: Git als begrenzte Quelle verwenden und mit weiteren Prozessquellen sowie Kontextwissen verbinden.

Geltungsgrenze: Aussagen über Git-Aktivität selbst sind selbstverständlich möglich, solange die Reichweite korrekt benannt wird.

G3

Die Zweckwanderung der Git-Daten

Muster: Zur Versionierung entstandene personenbezogene Daten werden ohne gesonderte Prüfung für Anwesenheitsüberwachung, individuelle Leistungsbeurteilung oder Personalentscheidungen verwendet.

Besser: Zweck und Rechtsgrundlage jeder Analyse prüfen; angemessene Zugriffsbeschränkungen, Datenminimierung und gegebenenfalls aggregierte Auswertung.

Geltungsgrenze: Projektbezogene Analysen können legitim sein; dies ist unabhängig von der Frage, ob die daraus abgeleitete Metrik methodisch brauchbar ist (E1).

G6

Die unbegrenzte Arbeitschronik

Muster: Vollständige personenbezogene Git-Historien werden ohne definierte Zwecke, Fristen, Zugriffsregeln oder Schutzmaßnahmen dauerhaft bewahrt.

Besser: Begründetes Aufbewahrungs- und Archivierungskonzept mit differenziertem Zugang, Schutzmaßnahmen und Berücksichtigung wissenschaftlicher wie persönlicher Interessen.

Geltungsgrenze: Wissenschaftliche Langzeitarchivierung kann zulässig und sinnvoll sein, verlangt aber geeignete rechtliche und organisatorische Rahmenbedingungen.

D2

Die vergessene Git-Vergangenheit

Muster: Nur der aktuelle Datenstand wird bereinigt, die frühere Git-Historie unverändert mit veröffentlicht.

Besser: Prüfung des gesamten freizugebenden Datenbestands einschließlich Historie; gegebenenfalls getrenntes Publikationsrepo.

Geltungsgrenze: Zulässige interne Bewahrung kann sinnvoll sein. History-Rewriting allein entfernt bereits verbreitete Kopien nicht zuverlässig.

C1

Der unentwirrbare Commit

Muster: Sachlich unabhängige Eingriffe werden ohne erkennbare gemeinsame Einheit in einem Commit zusammengefasst.

Besser: Logisch zusammengehörige Änderungsgruppen; gesonderte dokumentierte Massenoperationen.

Geltungsgrenze: Ein großer, sachlich zusammenhängender Datenimport ist legitim und gerade in Publikationsrepositories typisch.

C2

Die nichtssagende Versionsgeschichte

Muster: Nachrichten enthalten keinen hinreichenden Hinweis auf Gegenstand oder Zweck des Commits.

Besser: Kurze, aussagekräftige Nachrichten nach einer nachvollziehbaren Konvention.

Geltungsgrenze: Kurze oder automatisch erzeugte Commitnachrichten sind nicht an sich problematisch, wenn ihr Zweck ausreichend erkennbar bleibt.

C3

Die aufgeschobene Integration

Muster: Gemeinsam relevante Änderungen bleiben über lange Zeit in getrennten Branches und werden erst spät abgeglichen.

Besser: Regelmäßiger Abgleich und Integration entsprechend den tatsächlichen Abhängigkeiten der Arbeitspakete.

Geltungsgrenze: Langfristig getrennte Branches sind legitim, wenn Aufgaben unabhängig sind oder die Trennung bewusst begründet wird.

C4

Der Formatierungs-Tsunami

Muster: Editoren ändern bei fachlich kleinen Eingriffen großflächig Einrückungen, Whitespace oder Zeilenumbrüche.

Besser: Einheitliche, stabile Editor- und Formatierungskonventionen; reine Formatierungsänderungen getrennt committen.

Geltungsgrenze: Gezielte Formatmigrationen sind legitim; im Publikationsrepository kann die detaillierte Zeilenhistorie weniger wichtig sein.

C6

Die bewegliche Publikationsfassung

Muster: Nur die veränderliche Spitze eines Branches, etwa main, ist als veröffentlichte Fassung benannt.

Besser: Unveränderliche, eindeutig identifizierte Releases und wo geeignet persistente Identifikatoren; zusätzlich ein latest-Zugang.

Geltungsgrenze: Eine stets aktuelle Fassung ist nützlich, solange frühere Publikationsstände ebenfalls eindeutig referenzierbar sind.

Begriffe

Git

Software zur Verwaltung und zum Vergleich gespeicherter Bearbeitungsstände von Dateien.

Repository / Repo

Projektverzeichnis samt Git-Versionsgeschichte.

Commit

Ein von Git gespeicherter Projektstand einschließlich zugehöriger Metadaten.

Commit-Nachricht

Kurzer erklärender Text, der einem Commit beigefügt wird.

Push

Übertragen lokaler Commits an ein anderes Repository, zum Beispiel auf einem Server.

Pull

Holen und Integrieren von Änderungen aus einem anderen Repository in den lokalen Arbeitsstand.

Clone

Lokale Kopie eines Repositories, gewöhnlich einschließlich der erreichbaren Versionsgeschichte.

Branch

Benannter Entwicklungszweig, in dem unabhängig weitergearbeitet werden kann.

Merge

Technische Zusammenführung unterschiedlicher Entwicklungslinien.

Tag

Benannte Markierung eines bestimmten Commits, etwa für eine Veröffentlichung.

Diff

Darstellung der Unterschiede zwischen zwei Dateiversionen oder Projektständen.

Staging / Partial Staging

Vormerken aller oder ausgewählter Änderungen für den nächsten Commit.

Hook

Programm, das bei bestimmten Git-Vorgängen automatisch angestoßen wird.

CI/CD

Automatisierte Abläufe für Prüfungen, Verarbeitung und gegebenenfalls Bereitstellung von Ergebnissen.

GitHub / GitLab

Plattformen, die Git-Repositories hosten und um Zusammenarbeit und Automatisierung ergänzen.

Git Notes

Nachträglich hinzugefügte Anmerkungen zu Commits, ohne deren ursprünglichen Inhalt zu ändern.

Reflog

Lokale Aufzeichnung darüber, auf welche Commits bestimmte Git-Verweise zuletzt gezeigt haben.

Git LFS

Erweiterung, die große Dateiinhalte außerhalb der üblichen Git-Objektspeicherung verwaltet.

.gitattributes

Konfigurationsdatei für die Behandlung bestimmter Dateitypen durch Git.

.mailmap

Datei zur Vereinheitlichung verschiedener Autorenangaben in bestimmten Anzeigen und Statistiken.

TEI/XML

Strukturierte Textauszeichnung, häufig Grundlage digitaler wissenschaftlicher Editionen.

Relax NG / Schematron

Verfahren, mit denen sich XML-Dokumente gegen strukturelle beziehungsweise zusätzliche Regeln prüfen lassen.

XSLT

Sprache zur Umwandlung von XML-Daten in andere Darstellungen oder Formate.

ODD

Bei TEI ein Verfahren zur Spezifikation und Dokumentation eines projektspezifischen Auszeichnungsmodells.

Grundlagen: Git-Entdeckungsreise (83 Aussagen, 24 Begriffe) und Git Antipattern Academy (25 Karten). Alle Themen der beiden Sammlungen sind enthalten; Beziehungsgraphen und ausführliche Spielszenarien gehören nicht zu dieser kompakten Lesefassung.